

Complejidad Big-O – 8 Algoritmos

Alumno: Mariano Jose Garcia Servin

1. Acceso directo

```
1 def ultimo_elemento(lista):
2     if len(lista) == 0:
3         return None
4     return lista[-1]
5
6 print(ultimo_elemento([10, 20, 30, 40, 50]))
```

Complejidad: $O(1)$

Caso: Todos los casos.

Explicación: Solo buscamos el último elemento de la lista. No necesitamos recorrerla completa.

2. Búsqueda lineal

```
1 def busqueda_lineal(lista, objetivo):
2     for i in range(len(lista)):
3         if lista[i] == objetivo:
4             return i
5     return -1
6
7
8 print(busqueda_lineal([10, 25, 30, 45, 50], 30))
9 print(busqueda_lineal([10, 25, 30, 45, 50], 100))
```

Complejidad: $O(n)$

Caso: Peor caso.

Explicación: Vamos revisando los números uno por uno. Si el número está al final o no está, tenemos que revisar toda la lista.

3. Detección de duplicados

```

tarea de huillermos > lineal.py > ...
1  def hay_duplicados(lista):
2      n = len(lista)
3
4      for i in range(n):
5          for j in range(i + 1, n):
6              if lista[i] == lista[j]:
7                  return True
8
9      return False
10
11
12  print(hay_duplicados([1, 2, 3, 4, 5]))
13  print(hay_duplicados([1, 2, 3, 2, 5]))

```

Complejidad: $O(n^2)$

Caso: Peor caso.

Explicación: Usamos dos ciclos para comparar los números entre ellos. Por eso, cuando hay muchos números, se hacen muchas comparaciones.

4. División sucesiva

```

tarea de huillermos > lineal.py > ...
1  def divisiones_por_dos(n):
2      contador = 0
3
4      while n > 1:
5          n = n // 2
6          contador += 1
7
8      return contador
9
10
11  print(divisiones_por_dos(16))

```

Complejidad: $O(\log n)$

Caso: Peor caso.

Explicación: Dividimos el número entre 2 varias veces hasta llegar a 1. Como cada vez queda la mitad, necesitamos pocas vueltas.

5. Búsqueda binaria

```
lineal.py > ...
1 def busqueda_binaria(lista, objetivo):
2     inicio = 0
3     fin = len(lista) - 1
4
5     while inicio <= fin:
6         medio = (inicio + fin) // 2
7
8         if lista[medio] == objetivo:
9             return medio
10        elif lista[medio] < objetivo:
11            inicio = medio + 1
12        else:
13            fin = medio - 1
14
15    return -1
16
17
18 print(busqueda_binaria([10, 20, 30, 40, 50, 60, 70], 50))
19 print(busqueda_binaria([10, 20, 30, 40, 50, 60, 70], 100))
```

Complejidad: $O(\log n)$

Caso: Peor caso.

Explicación: En cada vuelta descartamos la mitad de los números. Así podemos encontrar el número sin revisar todos.

6. Fibonacci ineficiente

```
lineal.py > ...
1 def fibonacci(n):
2     if n <= 1:
3         return n
4
5     return fibonacci(n - 1) + fibonacci(n - 2)
6
7
8 print(fibonacci(6))
```

Complejidad: $O(2^n)$

Caso: Peor caso.

Explicación: La función se vuelve a llamar dos veces cada vez que se ejecuta. Esto hace que la cantidad de llamadas aumente muy rápido.

7. Procesamiento de dos listas

```
def procesar_listas(lista_n, lista_m):  
    suma = 0  
    for elemento in lista_n:  
        suma += elemento  
    producto = 1  
    for elemento in lista_m:  
        producto *= elemento  
    return suma, producto  
  
print(procesar_listas([1, 2, 3, 4], [2, 3, 4]))
```

Complejidad: $O(N + M)$

Caso: Peor caso.

Explicación: Primero recorremos toda la lista N y después toda la lista M. Por eso sumamos los dos recorridos.